

R-Code Beispiele aus WDDA Vorlesung 02

WDDA – Datenmanagement und Datenanalyse

Prof. Dr. Ulrich Matter

2026-04-29

Inhaltsverzeichnis

1	Einleitung	1
2	Vorbereitung	2
3	Häufigkeitsverteilungen	2
4	Relative Häufigkeiten	3
5	Grafische Darstellungen	3
5.1	Balkendiagramme	3
5.2	Kuchendiagramme	5
6	Zusammenfassen quantitativer Daten	7
7	Kumulierte Häufigkeiten	8
8	Histogramme	8
9	Schiefte	11
10	Filterung von Datensätzen	13
11	Kreuztabellen	15
12	Recoding und Umgestalten	17
13	Aggregation	18
14	Streudiagramme	19

1 Einleitung

Dieses Dokument fasst alle R-Code-Beispiele der WDDA Folien aus Vorlesung 2 zusammen. Der Fokus liegt auf der deskriptiven Statistik, insbesondere auf tabellarischen und grafischen Darstellungen von Daten. Die hier verwendeten Daten finden Sie in `data/WDDA_02.xlsx`.

Die deskriptive Statistik ist ein grundlegender Bereich der Statistik, der sich mit der Zusammenfassung, Visualisierung und Interpretation von Daten befasst. Im Gegensatz zur inferentiellen Statistik, die Schlussfolgerungen über eine Grundgesamtheit auf Basis von Stichproben zieht, beschreibt die deskriptive Statistik die vorliegenden Daten ohne darüber hinausgehende Schlussfolgerungen.

2 Vorbereitung

```
# Laden der benötigten Pakete
library(readxl) # Paket zum Einlesen von Excel-Dateien

# Laden der Datensätze
# (Auto, Variable "Brand")
Brand <- read_excel("../data/WDDA_02.xlsx", sheet = "Auto")

# (Audit, Variable "Time")
Time <- read_excel("../data/WDDA_02.xlsx", sheet = "Audit")
# Convert Time to numeric vector
# unlist() wandelt die Datenstruktur in einen Vektor um
# as.numeric() konvertiert die Werte in numerische Werte
Time <- as.numeric(unlist(Time))

# (Restaurant)
Restaurant <- read_excel("../data/WDDA_02.xlsx", sheet = "Restaurant")

# (Inventory, Variablen "shirt", "price")
Inventory <- read_excel("../data/WDDA_02.xlsx", sheet = "Inventory")

# (Stereo, Variablen "Week", "Commercials", "Sales")
Stereo <- read_excel("../data/WDDA_02.xlsx", sheet = "Stereo")
```

Hinweis zu R-Paketen: In R werden zusätzliche Funktionalitäten durch Pakete bereitgestellt. Das Paket `readxl` ermöglicht das Einlesen von Excel-Dateien. Pakete müssen zunächst mit `install.packages("paketname")` installiert und dann mit `library(paketname)` geladen werden.

Hinweis zu Datenpfaden: Beachten Sie, dass in den R-Markdown-Dateien relative Pfade verwendet werden (`../data/`), während im R-Skript direkte Pfade (`data/`) verwendet werden. Dies liegt daran, dass R-Markdown-Dateien vom Speicherort der Datei aus arbeiten, während R-Skripte vom Arbeitsverzeichnis aus arbeiten.

3 Häufigkeitsverteilungen

Eine Häufigkeitsverteilung fasst die Daten tabellarisch zusammen, indem sie die Anzahl der Elemente in nicht-überlappenden Klassen angibt.

```
# Eindeutige Werte ermitteln
unique_brands <- Brand |> unique() # Alternative Schreibweise: unique(Brand)

# Häufigkeiten zählen
freq <- Brand |> table() # Die table()-Funktion erstellt eine Häufigkeitstabelle
print(freq) # Ausgabe der Häufigkeitstabelle
```

```
## Brand
##      Audi      BMW Mercedes      Opel      VW
##         8         5         13         5         19
```

Statistische Erklärung: Eine Häufigkeitsverteilung ist eine grundlegende Methode zur Zusammenfassung von Daten. Sie zeigt, wie oft jeder Wert oder jede Kategorie in einem Datensatz vorkommt. Bei kategorialen

Daten (wie Automarken) ist dies besonders nützlich, um einen Überblick über die Verteilung zu erhalten.

R-Erklärung:

- Die Pipe-Operator `|>` in R (eingeführt in R 4.1.0) leitet das Ergebnis des linken Ausdrucks als erstes Argument an die rechte Funktion weiter.
- Die Funktion `unique()` gibt alle eindeutigen Werte in einem Vektor zurück.
- Die Funktion `table()` erstellt eine Häufigkeitstabelle, die für jede eindeutige Kategorie die Anzahl der Vorkommen zählt.

4 Relative Häufigkeiten

Die relative Häufigkeit einer Kategorie entspricht dem Anteil der absoluten Häufigkeit an der Gesamthäufigkeit.

```
# Berechnung der relativen Häufigkeiten
relfreq <- freq / sum(freq)
print(relfreq) # Ausgabe der relativen Häufigkeiten
```

```
## Brand
##      Audi      BMW Mercedes      Opel      VW
##      0.16      0.10      0.26      0.10      0.38
```

```
# Alternative Methode mit prop.table()
relfreq_alt <- prop.table(freq)
print(relfreq_alt) # Gibt das gleiche Ergebnis wie oben
```

```
## Brand
##      Audi      BMW Mercedes      Opel      VW
##      0.16      0.10      0.26      0.10      0.38
```

Statistische Erklärung: Relative Häufigkeiten geben den Anteil jeder Kategorie an der Gesamtmenge an und werden als Dezimalzahlen oder Prozentsätze ausgedrückt. Sie sind besonders nützlich für Vergleiche zwischen Datensätzen unterschiedlicher Grösse. Die Summe aller relativen Häufigkeiten ergibt immer 1 (oder 100%).

R-Erklärung:

- Die Division einer Häufigkeitstabelle durch ihre Summe (`freq / sum(freq)`) berechnet die relativen Häufigkeiten.
- Die Funktion `prop.table()` ist eine spezialisierte Funktion in R, die direkt relative Häufigkeiten aus einer Häufigkeitstabelle berechnet.
- Beide Methoden liefern das gleiche Ergebnis, aber `prop.table()` ist oft kürzer und lesbarer.

5 Grafische Darstellungen

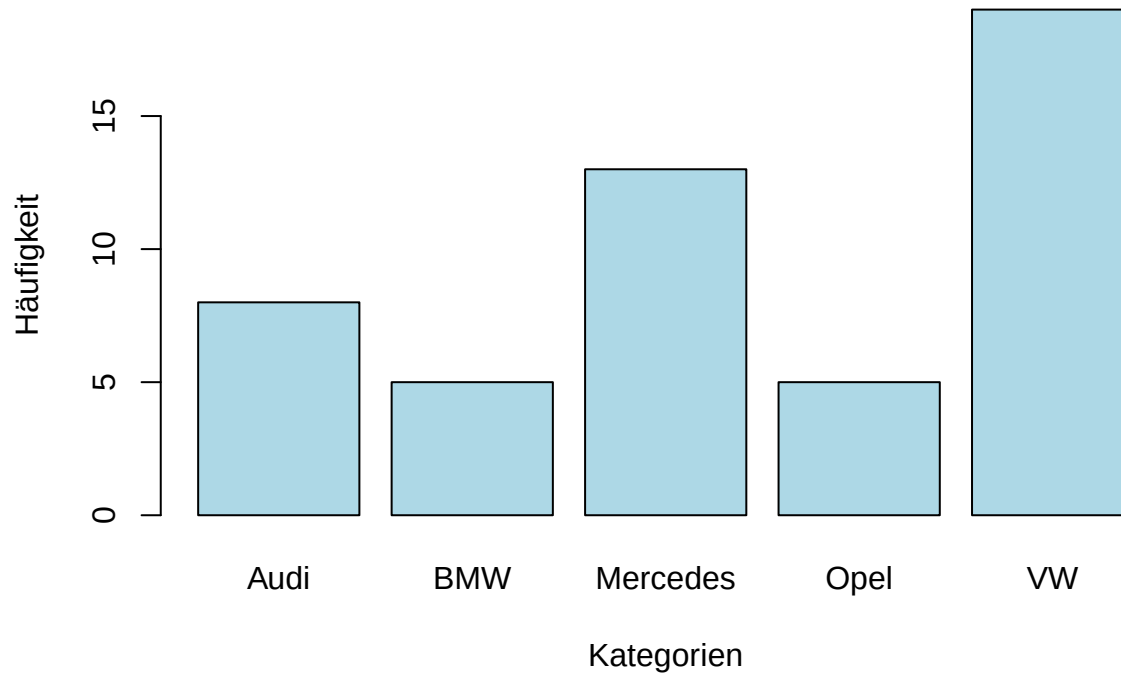
5.1 Balkendiagramme

Balkendiagramme visualisieren die absolute Häufigkeit von kategorialen Daten.

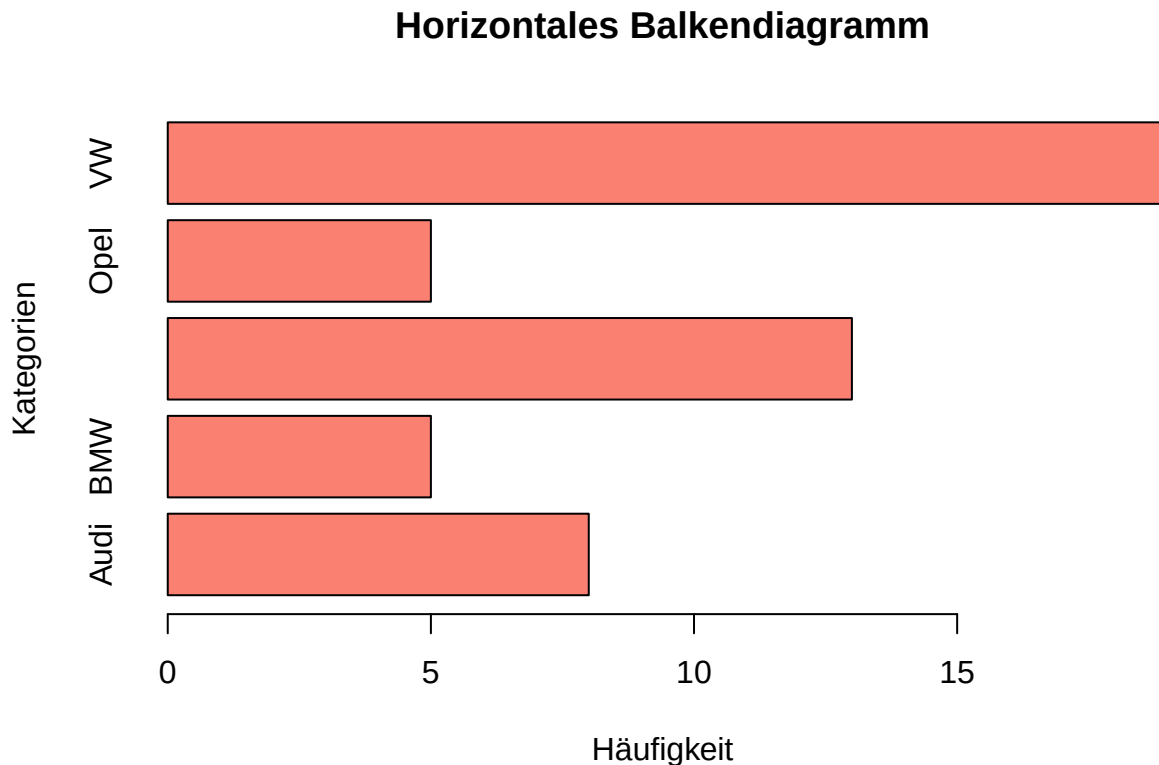
```
# Erstellen eines Balkendiagramms
barplot(freq,
  main = "Balkendiagramm der Häufigkeiten", # Titel des Diagramms
  col = "lightblue",                       # Farbe der Balken
```

```
xlab = "Kategorien",           # Beschriftung der x-Achse
ylab = "Häufigkeit")          # Beschriftung der y-Achse
```

Balkendiagramm der Häufigkeiten



```
# Erstellung eines horizontalen Balkendiagramms
barplot(freq,
  horiz = TRUE,           # Horizontale Ausrichtung
  main = "Horizontales Balkendiagramm",
  col = "salmon",
  xlab = "Häufigkeit",
  ylab = "Kategorien")
```



Statistische Erklärung: Balkendiagramme sind eine der häufigsten Methoden zur Visualisierung kategorialer Daten. Jeder Balken repräsentiert eine Kategorie, und die Höhe (oder Länge bei horizontalen Diagrammen) entspricht der Häufigkeit. Balkendiagramme eignen sich besonders gut für nominale Daten (Kategorien ohne natürliche Reihenfolge) und ordinale Daten (Kategorien mit einer natürlichen Reihenfolge).

R-Erklärung:

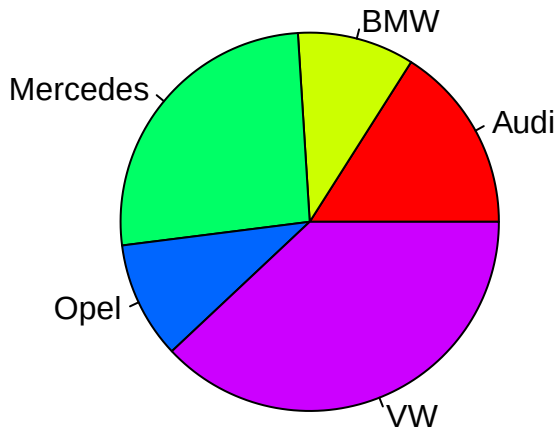
- Die Funktion `barplot()` erstellt ein Balkendiagramm aus einer Häufigkeitstabelle oder einem numerischen Vektor.
- Mit dem Parameter `horiz = TRUE` kann ein horizontales Balkendiagramm erstellt werden.
- Weitere wichtige Parameter sind:
 - `main`: Titel des Diagramms
 - `col`: Farbe der Balken (R unterstützt viele Farbnamen und RGB-Werte)
 - `xlab` und `ylab`: Beschriftungen der Achsen
 - `names.arg`: Alternative Beschriftungen für die Kategorien

5.2 Kuchendiagramme

Kreisdiagramme visualisieren die relative Häufigkeit von nominalen Daten.

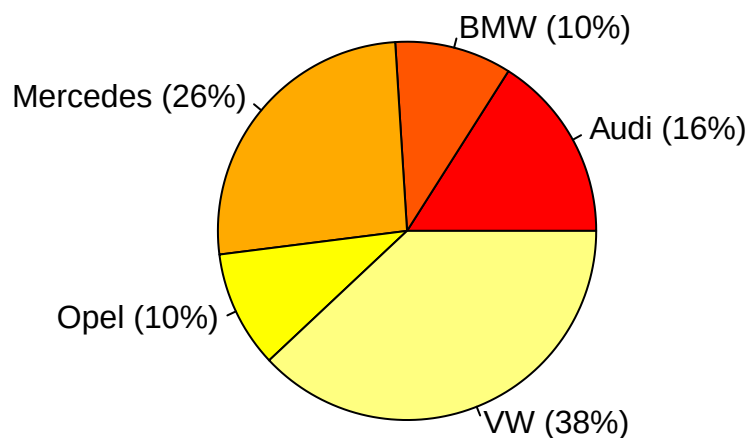
```
# Erstellen eines Kreisdiagramms
pie(freq,
    main = "Kuchendiagramm der relativen Häufigkeiten",
    col = rainbow(length(freq))) # Verwendung verschiedener Farben
```

Kuchendiagramm der relativen Häufigkeiten



```
# Kreisdiagramm mit Prozentangaben
pie(freq,
     main = "Kuchendiagramm mit Prozentangaben",
     labels = paste0(names(freq), " (", round(100*relfreq, 1), "%)"),
     col = heat.colors(length(freq)))
```

Kuchendiagramm mit Prozentangaben



Statistische Erklärung: Kreisdiagramme (auch Kuchendiagramme genannt) stellen die relativen Anteile verschiedener Kategorien als Sektoren eines Kreises dar. Die Grösse jedes Sektors ist proportional zum relativen Anteil der entsprechenden Kategorie. Kreisdiagramme eignen sich am besten für nominale Daten mit wenigen Kategorien (idealerweise nicht mehr als 5-7), da zu viele Sektoren die Lesbarkeit beeinträchtigen können.

R-Erklärung:

- Die Funktion `pie()` erstellt ein Kreisdiagramm aus einem numerischen Vektor.
- Mit `rainbow()`, `heat.colors()` und ähnlichen Funktionen können automatisch Farbpaletten erstellt werden.

- Der Parameter `labels` ermöglicht benutzerdefinierte Beschriftungen für die Sektoren.
- Die Funktion `paste0()` verbindet Zeichenketten ohne Trennzeichen, während `paste()` standardmässig Leerzeichen einfügt.

Hinweis zur Visualisierung: Obwohl Kreisdiagramme weit verbreitet sind, werden sie in der professionellen Datenanalyse oft kritisch gesehen, da Menschen Winkel und Flächen schwerer vergleichen können als Längen. Balkendiagramme sind daher oft die bessere Wahl für präzise Vergleiche.

6 Zusammenfassen quantitativer Daten

Bei quantitativen Daten mit vielen einzigartigen Werten ist eine Gruppierung in Klassen (Bins) sinnvoll.

```
# Häufigkeitstabelle für ungegrupperte Daten
table(Time)

## Time
## 12 13 14 15 16 17 18 19 20 21 22 23 27 28 33
##  1  1  2  2  1  1  3  1  1  1  2  1  1  1  1

# Definieren von Klassen (Bins)
bins <- c(0, 14 + 5 * (0:5)) # Erweitert auf 0, 14, 19, 24, 29, 34, 39
print(bins) # Zeigt die definierten Klassengrenzen an

## [1]  0 14 19 24 29 34 39

# Gruppierung der Daten
Time_binned <- Time |> cut(bins)
table(Time_binned)

## Time_binned
##  (0,14] (14,19] (19,24] (24,29] (29,34] (34,39]
##      4      8      5      2      1      0
```

Statistische Erklärung: Bei kontinuierlichen oder diskreten Daten mit vielen verschiedenen Werten ist es oft sinnvoll, die Daten in Klassen (Bins) zu gruppieren. Dies vereinfacht die Darstellung und Interpretation der Daten. Die Wahl der Klassenanzahl und -breite ist ein wichtiger Schritt, der die Aussagekraft der Analyse beeinflussen kann:

- Zu wenige Klassen können wichtige Muster verbergen
- Zu viele Klassen können zu einer fragmentierten Darstellung führen, die schwer zu interpretieren ist

Eine Faustregel ist die Sturges-Regel: $k = 1 + 3.322 \times \log_{10}(n)$, wobei k die Anzahl der Klassen und n die Stichprobengröße ist.

R-Erklärung:

- Die Funktion `cut()` teilt numerische Daten in Intervalle (Bins) ein.
- Der erste Parameter ist der zu gruppierende Vektor.
- Der zweite Parameter gibt die Grenzen der Intervalle an.
- Das Ergebnis ist ein Faktor, dessen Levels die Intervalle darstellen.
- Die Notation `(a,b]` bedeutet, dass das Intervall a ausschliesst und b einschliesst.

7 Kumulierte Häufigkeiten

Die kumulierte Häufigkeit gibt an, wie viele Datenwerte kleiner oder gleich der oberen Grenze der jeweiligen Klasse sind.

```
# Gesamtsumme der Werte in 'Time' (nur als Beispiel, nicht immer sinnvoll)
sum(Time)
```

```
## [1] 385
```

```
# Erstellen der kumulierten Häufigkeitstabelle
cum_freq <- Time_binned |>
  table() |>
  cumsum()
print(cum_freq) # Ausgabe der kumulierten Häufigkeiten
```

```
## (0,14] (14,19] (19,24] (24,29] (29,34] (34,39]
##      4      12      17      19      20      20
```

```
# Berechnung der kumulierten relativen Häufigkeiten
cum_rel_freq <- cum_freq / sum(table(Time_binned))
print(cum_rel_freq) # Ausgabe der kumulierten relativen Häufigkeiten
```

```
## (0,14] (14,19] (19,24] (24,29] (29,34] (34,39]
##  0.20  0.60  0.85  0.95  1.00  1.00
```

Statistische Erklärung: Kumulierte Häufigkeiten summieren die Häufigkeiten bis zu einem bestimmten Punkt auf. Sie sind nützlich, um zu bestimmen, wie viele Beobachtungen unter einem bestimmten Wert liegen. Kumulierte relative Häufigkeiten geben diesen Anteil als Proportion (zwischen 0 und 1) an.

Kumulierte Häufigkeiten werden oft verwendet für:

- Die Berechnung von Perzentilen und Quartilen
- Die Erstellung von kumulativen Verteilungsfunktionen
- Die Analyse von Schwellenwerten in Daten

R-Erklärung:

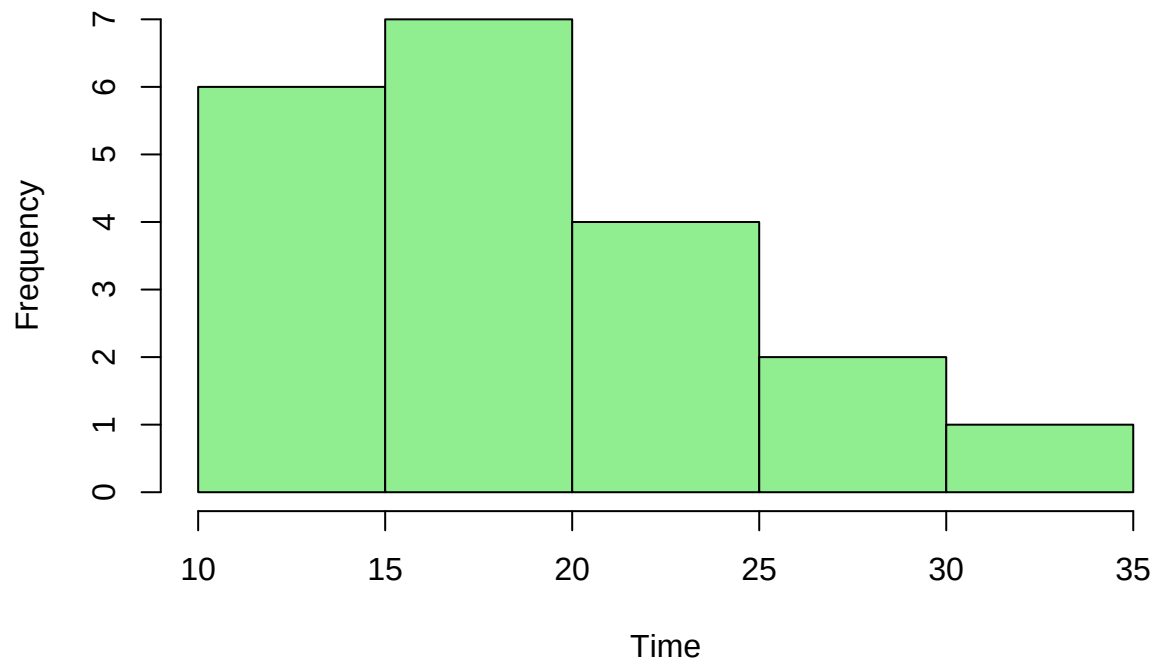
- Die Funktion `cumsum()` berechnet die kumulierte Summe eines Vektors.
- In der Pipe-Kette wird zuerst eine Häufigkeitstabelle erstellt und dann die kumulierte Summe berechnet.
- Die Division durch die Gesamtsumme ergibt die kumulierten relativen Häufigkeiten.

8 Histogramme

Histogramme sind die wichtigste grafische Darstellung für quantitative Daten.

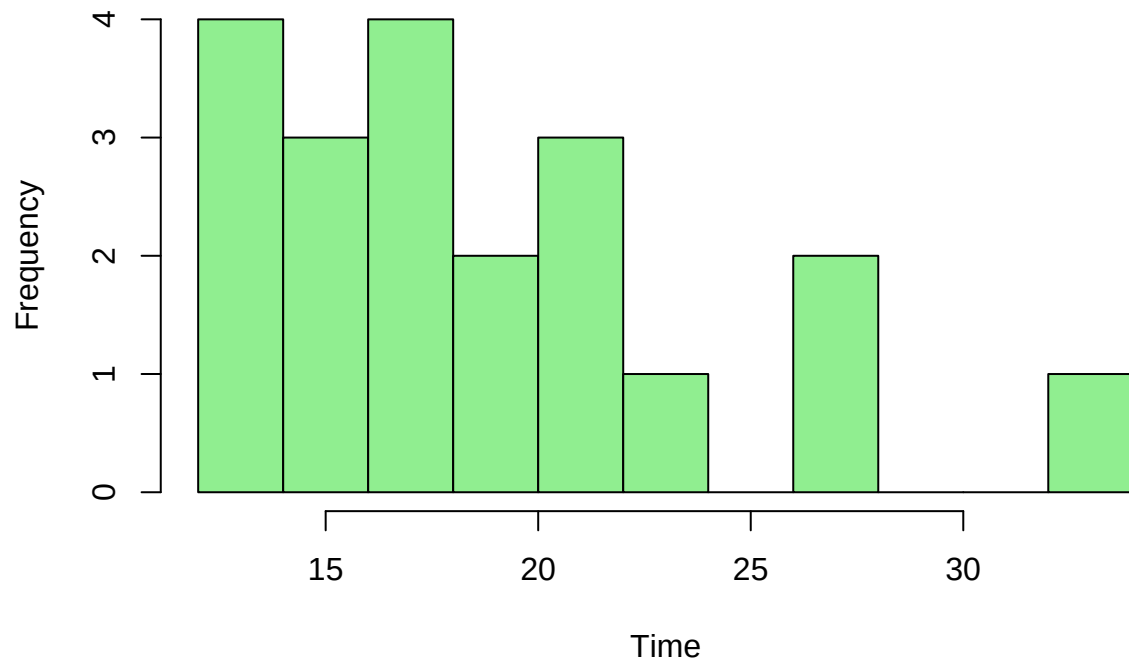
```
# Standard-Histogramm
hist(Time,
  main = "Histogramm von Time",
  xlab = "Time",
  col = "lightgreen")
```


Histogramm von Time



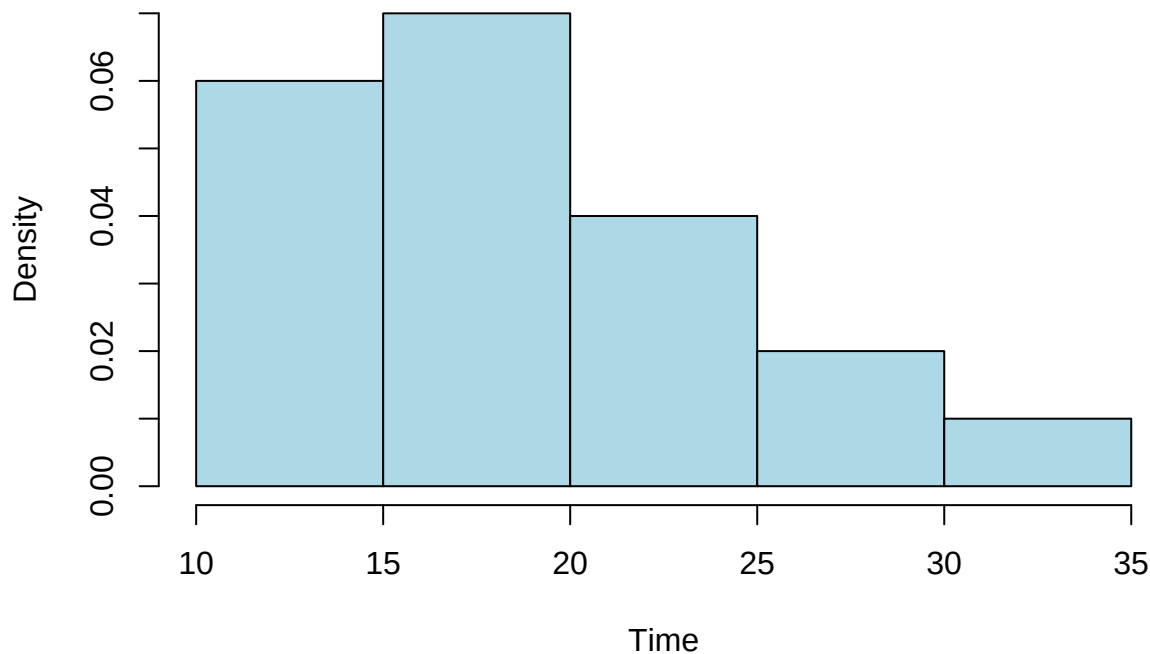
```
# Histogramm mit angepasster Klassenanzahl  
hist(Time,  
      breaks = 10,  
      main = "Histogramm von Time (10 Klassen)",  
      xlab = "Time",  
      col = "lightgreen")
```

Histogramm von Time (10 Klassen)



```
# Histogramm mit Wahrscheinlichkeitsdichte statt Häufigkeiten
hist(Time,
      freq = FALSE, # Zeigt Dichte statt Häufigkeiten
      main = "Histogramm mit Wahrscheinlichkeitsdichte",
      xlab = "Time",
      col = "lightblue")
```

Histogramm mit Wahrscheinlichkeitsdichte



Statistische Erklärung: Histogramme sind eine grundlegende grafische Darstellung für kontinuierliche Daten. Sie teilen den Wertebereich in Intervalle (Bins) und zeigen die Häufigkeit oder Dichte der Beobachtungen in jedem Intervall. Histogramme helfen dabei:

- Die Form der Verteilung zu erkennen (symmetrisch, rechtsschief, linksschief, bimodal, etc.)
- Ausreisser zu identifizieren
- Die Streuung und zentrale Tendenz der Daten visuell zu erfassen

Die Wahl der Klassenanzahl ist entscheidend: Zu wenige Klassen können wichtige Muster verbergen, zu viele können zu einer verrauschten Darstellung führen.

R-Erklärung:

- Die Funktion `hist()` erstellt ein Histogramm aus einem numerischen Vektor.
- Der Parameter `breaks` kontrolliert die Anzahl oder Position der Klassenintervalle:
 - Eine einzelne Zahl gibt die ungefähre Anzahl der gewünschten Klassen an
 - Ein Vektor gibt die exakten Klassengrenzen an
- Mit `freq = FALSE` wird die Wahrscheinlichkeitsdichte statt der absoluten Häufigkeit dargestellt.
- Die Fläche unter einem Dichte-Histogramm summiert sich zu 1.

Tipp: Bei der Wahl der Klassenanzahl gilt es, das Signal (Muster) zu modellieren und nicht das Rauschen. Experimentieren Sie mit verschiedenen Werten für `breaks`, um die aussagekräftigste Darstellung zu finden.

9 Schiefe

Histogramme können Hinweise auf die Schiefe (Asymmetrie) der Verteilung geben.

```
# Berechnung der Schiefe mit dem Paket 'e1071'
if (!requireNamespace("e1071", quietly = TRUE)) {
  install.packages("e1071")
}
```

```
library(e1071)
skewness_value <- skewness(Time)
print(skewness_value)

## [1] 0.844531

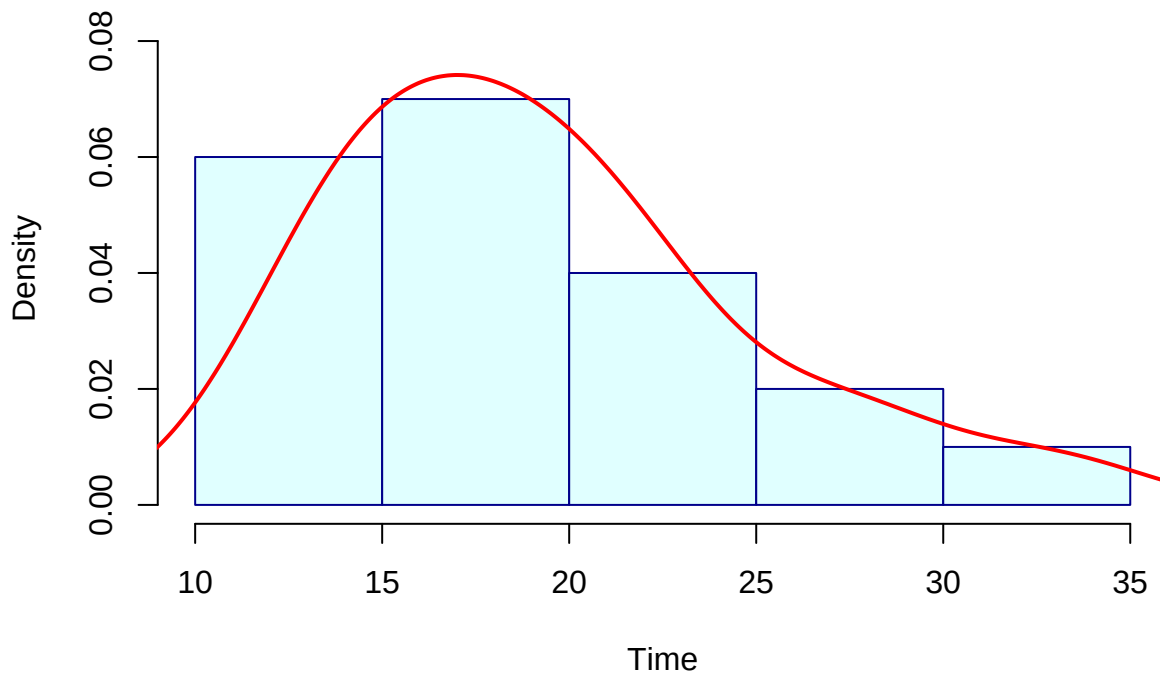
# Visualisierung der Schiefe mit einem Histogramm und einer Dichtefunktion
# Berechne zuerst die Dichte, um die maximale Höhe zu bestimmen
dens <- density(Time)
y_max <- max(c(max(dens$y), max(hist(Time, freq = FALSE, plot = FALSE)$density)))

## Warning in hist.default(Time, freq = FALSE, plot = FALSE): argument 'freq' is
## not made use of

# Erstelle das Histogramm mit angepasstem y-Limit
hist(Time,
      freq = FALSE,
      main = "Histogramm mit Dichtefunktion",
      xlab = "Time",
      col = "lightcyan",
      border = "darkblue",
      ylim = c(0, y_max * 1.1)) # 10% Puffer über dem Maximum

# Hinzufügen einer Dichtefunktion
lines(dens, col = "red", lwd = 2)
```

Histogramm mit Dichtefunktion



Statistische Erklärung: Die Schiefe (Skewness) ist ein Mass für die Asymmetrie einer Verteilung. Sie

beschreibt, ob die Verteilung symmetrisch ist oder ob sie einen längeren Schwanz auf einer Seite hat:

- **Positive Schiefe (> 0):** Der rechte Schwanz ist länger; die meisten Werte liegen links vom Mittelwert (rechtsschief)
- **Negative Schiefe (< 0):** Der linke Schwanz ist länger; die meisten Werte liegen rechts vom Mittelwert (linksschief)
- **Keine Schiefe ($= 0$):** Die Verteilung ist symmetrisch (wie bei einer Normalverteilung)

Die Schiefe ist wichtig für die Wahl geeigneter statistischer Methoden, da viele Verfahren eine symmetrische Verteilung voraussetzen.

R-Erklärung:

- Die Funktion `skewness()` aus dem Paket `e1071` berechnet die Schiefe eines numerischen Vektors.
- Die Funktion `density()` schätzt die Wahrscheinlichkeitsdichtefunktion der Daten.
- Mit `lines()` kann eine Linie zu einem bestehenden Plot hinzugefügt werden.
- Die Parameter `col` und `lwd` steuern die Farbe und Breite der Linie.

10 Filterung von Datensätzen

Filter ermöglichen die Auswahl von Teilmengen eines Datensatzes anhand von Bedingungen.

```
library(dplyr) # Laden des dplyr-Pakets für Datenmanipulation
```

```
##
## Attaching package: 'dplyr'
##
## The following objects are masked from 'package:stats':
##
##     filter, lag
##
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
# Auswahl der 'Price'-Spalte für Restaurants mit guter Qualität
good_price <-
  Restaurant |>
  filter(Quality == 'Good') |> # Filtert Zeilen, wo Quality gleich 'Good' ist
  select(Price)                # Wählt nur die Price-Spalte aus

print(good_price)
```

```
## # A tibble: 150 x 1
##   Price
##   <dbl>
## 1    22
## 2    33
## 3    19
## 4    11
## 5    23
## 6    33
## 7    44
## 8    30
## 9    33
## 10   22
```

```
## # i 140 more rows
```

```
# Anzahl der Restaurants mit guter Qualität
num_good <- Restaurant |> filter(Quality == 'Good') |> nrow()
print(num_good)
```

```
## [1] 150
```

```
# Anzahl der Restaurants mit Price <= 20
num_cheap <- Restaurant |> filter(Price <= 20) |> nrow()
print(num_cheap)
```

```
## [1] 99
```

```
# Anzahl der Restaurants mit guter Qualität UND Price <= 20
num_good_and_cheap <- Restaurant |> filter(Quality == 'Good' & Price <= 20) |> nrow()
print(num_good_and_cheap)
```

```
## [1] 43
```

```
# Anzahl der Restaurants mit guter Qualität ODER Price <= 20
num_good_or_cheap <- Restaurant |> filter(Quality == 'Good' | Price <= 20) |> nrow()
print(num_good_or_cheap)
```

```
## [1] 206
```

```
# Beispiel für komplexere Filterung mit mehreren Bedingungen
complex_filter <- Restaurant |>
  filter((Quality == 'Good' | Quality == 'Excellent') & Price < 30) |>
  arrange(Price) # Sortiert nach Preis
head(complex_filter)
```

```
## # A tibble: 6 x 3
##   Restaurant Quality Price
##   <dbl> <chr> <dbl>
## 1      53 Good     10
## 2       8 Good     11
## 3     169 Good     11
## 4     186 Good     11
## 5     188 Good     11
## 6      69 Good     12
```

Statistische Erklärung: Die Filterung von Daten ist ein grundlegender Schritt in der explorativen Datenanalyse. Sie ermöglicht es, bestimmte Teilmengen eines Datensatzes zu isolieren und separat zu analysieren. Dies ist besonders nützlich, um:

- Hypothesen über Untergruppen zu testen
- Beziehungen zwischen Variablen in spezifischen Segmenten zu untersuchen
- Daten für bestimmte Analysen vorzubereiten

R-Erklärung:

- Das Paket `dplyr` ist Teil der Tidyverse-Sammlung und bietet leistungsstarke Funktionen zur Datenmanipulation.

- Die Funktion `filter()` wählt Zeilen basierend auf Bedingungen aus.
- Logische Operatoren in R:
 - `==`: Gleichheit
 - `!=`: Ungleichheit
 - `<`, `<=`, `>`, `>=`: Vergleichsoperatoren
 - `&`: Logisches UND (beide Bedingungen müssen erfüllt sein)
 - `|`: Logisches ODER (mindestens eine Bedingung muss erfüllt sein)
 - `!`: Logische Negation
- Die Funktion `select()` wählt Spalten aus.
- Die Funktion `arrange()` sortiert die Daten nach einer oder mehreren Spalten.
- Die Funktion `nrow()` gibt die Anzahl der Zeilen zurück.

11 Kreuztabellen

Kreuztabellen (Kontingenztafeln) fassen die Daten für zwei Variablen tabellarisch zusammen.

```
# Extrahieren der Quality und Price Variablen aus dem Restaurant Datensatz
Quality <- Restaurant$Quality
Price <- Restaurant$Price

# Einfache Kreuztabelle
ct1 <- table(Quality, Price)
print(ct1)
```

```
##           Price
## Quality
## Disappointing  6  4  3  3  2  4  4  5  5  6 10  1  3  5  1  3  4  5  5  3  0
## Excellent      0  0  0  1  0  0  0  1  0  0  2  1  1  2  1  1  1  3  2  0  4
## Good           1  4  3  5  6  1  5  3  3  3  9  9  4  6  5  9  4  8 10  0  7
##           Price
## Quality
## Disappointing  0  0  1  0  0  0  1  0  0  0  0  0  0  0  0  0  0  0  0  0
## Excellent      3  5  3  2  3  4  2  2  4  4  2  1  2  2  3  2  2
## Good           5  6  6  5  5  2  4  6  1  0  2  0  1  1  0  0  1
```

```
# Kreuztabelle mit gruppierten Preisen
bins <- 10 * (0:5)
Price_ranges <- Price |> cut(bins)
# Erstellen einer Kreuztabelle mit den Preisbereichen
ct2 <- table(Quality, Price_ranges)
print(ct2)
```

```
##           Price_ranges
## Quality      (0,10] (10,20] (20,30] (30,40] (40,50]
## Disappointing      6      46      30       2       0
## Excellent          0       4      16      28      18
## Good              1      42      62      40       5
```

```
# Berechnung der relativen Häufigkeiten (zeilenweise)
prop.table(ct2, margin = 1) # margin = 1 bedeutet zeilenweise
```

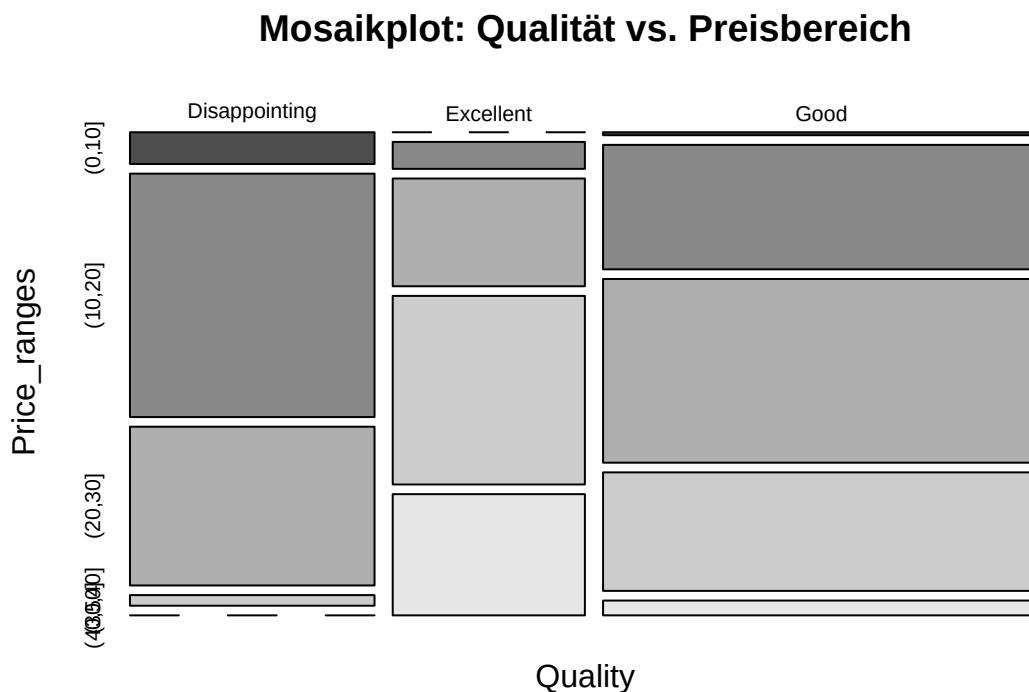
```
##           Price_ranges
```

```
## Quality          (0,10]   (10,20]   (20,30]   (30,40]   (40,50]
## Disappointing 0.071428571 0.547619048 0.357142857 0.023809524 0.000000000
## Excellent     0.000000000 0.060606061 0.242424242 0.424242424 0.272727273
## Good          0.006666667 0.280000000 0.413333333 0.266666667 0.033333333
```

```
# Berechnung der relativen Häufigkeiten (spaltenweise)
prop.table(ct2, margin = 2) # margin = 2 bedeutet spaltenweise
```

```
##          Price_ranges
## Quality          (0,10]   (10,20]   (20,30]   (30,40]   (40,50]
## Disappointing 0.85714286 0.50000000 0.27777778 0.02857143 0.00000000
## Excellent     0.00000000 0.04347826 0.14814815 0.40000000 0.78260870
## Good          0.14285714 0.45652174 0.57407407 0.57142857 0.21739130
```

```
# Visualisierung der Kreuztabelle als Mosaikplot
mosaicplot(ct2,
  main = "Mosaikplot: Qualität vs. Preisbereich",
  color = TRUE)
```



Statistische Erklärung: Kreuztabellen (auch Kontingenztafeln oder Häufigkeitstabellen mit zwei Eingängen genannt) zeigen die gemeinsame Häufigkeitsverteilung zweier kategorialer Variablen. Sie sind ein wichtiges Werkzeug, um Beziehungen zwischen kategorialen Variablen zu untersuchen:

- Die Zeilen repräsentieren die Kategorien der ersten Variable
- Die Spalten repräsentieren die Kategorien der zweiten Variable
- Jede Zelle enthält die Anzahl der Beobachtungen, die zu beiden Kategorien gehören

Kreuztabellen können verwendet werden, um:

- Zusammenhänge zwischen kategorialen Variablen zu identifizieren
- Bedingte Verteilungen zu analysieren
- Chi-Quadrat-Tests auf Unabhängigkeit durchzuführen

R-Erklärung:

- Die Funktion `table()` erstellt eine Kreuztabelle aus zwei oder mehr Vektoren.
- Mit `prop.table()` können relative Häufigkeiten berechnet werden:
 - `margin = 1`: Zeilenweise relative Häufigkeiten (Summe jeder Zeile = 1)
 - `margin = 2`: Spaltenweise relative Häufigkeiten (Summe jeder Spalte = 1)
 - Ohne `margin`: Gesamtrelative Häufigkeiten (Summe aller Zellen = 1)
- Die Funktion `mosaicplot()` erstellt eine grafische Darstellung einer Kreuztabelle, bei der die Flächen proportional zu den Häufigkeiten sind.

12 Recoding und Umgestalten

Beim Recoding werden Variablen umkodiert oder neu strukturiert.

```
library(stringr) # Paket für String-Manipulation

# Aufteilen einer Variable
shirt_split <- Inventory |>
  pull(shirt) |>           # Extrahiert die shirt-Spalte als Vektor
  str_split_fixed(',', 3)  # Teilt jeden String an Kommas in 3 Teile

# Anzeigen der ersten Zeilen des Ergebnisses
head(shirt_split)

##      [,1]      [,2]      [,3]
## [1,] "T-shirt"  "blue"    "S"
## [2,] "T-shirt"  "white"   "M"
## [3,] "T-shirt"  "white"   "S"
## [4,] "Polo shirt" "blue"    "XS"
## [5,] "T-shirt"  "green"   "L"
## [6,] "T-shirt"  "red"     "XS"

# Erstellen eines neuen Dataframes
Inventory2 <- shirt_split |>
  data.frame() |>           # Konvertiert die Matrix in einen Dataframe
  setNames(c('style', 'colour', 'size')) # Benennt die Spalten

# Hinzufügen der Preise als neue Spalte
Inventory2$price <- Inventory |> pull(price)

# Berechnen eines 30%-Rabattes auf den Preis und Hinzufügen als neue Spalte 'discount'
Inventory2$discount <- Inventory2$price * (1 - 0.30)

# Überprüfe die Struktur des neuen Dataframes
str(Inventory2)

## 'data.frame':    200 obs. of  5 variables:
## $ style      : chr  "T-shirt" "T-shirt" "T-shirt" "Polo shirt" ...
## $ colour     : chr  "blue" "white" "white" "blue" ...
## $ size       : chr  "S" "M" "S" "XS" ...
## $ price      : num  75 115 105 75 100 70 85 75 105 115 ...
## $ discount   : num  52.5 80.5 73.5 52.5 70 49 59.5 52.5 73.5 80.5 ...
```

```
# Beispiel für Umkodierung einer kategorialen Variable
# Erstellen einer neuen Spalte 'price_category' basierend auf dem Preis
Inventory2$price_category <- cut(Inventory2$price,
                                breaks = c(0, 20, 40, 60, Inf),
                                labels = c("Günstig", "Mittel", "Teuer", "Premium"))

# Anzeigen der Häufigkeiten der Preiskategorien
table(Inventory2$price_category)
```

```
##
##   Günstig   Mittel   Teuer   Premium
##         0         0         0        200
```

Statistische Erklärung: Recoding (Umkodierung) ist ein wichtiger Schritt in der Datenaufbereitung, bei dem Variablen transformiert oder neu strukturiert werden. Dies kann verschiedene Zwecke erfüllen:

- **Aufteilen von Variablen:** Eine Variable, die mehrere Informationen enthält, wird in separate Variablen aufgeteilt
- **Kategorisierung:** Kontinuierliche Variablen werden in Kategorien umgewandelt
- **Umkodierung:** Ändern von Werten oder Labels für eine klarere Interpretation
- **Berechnung abgeleiteter Variablen:** Erstellen neuer Variablen basierend auf bestehenden

Diese Transformationen sind oft notwendig, um Daten für spezifische Analysen vorzubereiten oder um die Interpretation zu erleichtern.

R-Erklärung:

- Das Paket `stringr` bietet leistungsstarke Funktionen zur Manipulation von Zeichenketten.
- Die Funktion `pull()` aus `dplyr` extrahiert eine Spalte als Vektor.
- `str_split_fixed()` teilt Zeichenketten an einem bestimmten Trennzeichen und gibt eine Matrix mit fester Anzahl von Spalten zurück.
- `data.frame()` konvertiert eine Matrix in einen Dataframe.
- `setNames()` benennt die Spalten eines Dataframes um.
- Der `$`-Operator wird verwendet, um auf Spalten zuzugreifen oder neue Spalten zu erstellen.

13 Aggregation

Aggregation fasst Daten zusammen.

```
# Aggregieren nach der Variable 'colour'
agg_result <- Inventory2 |>
  aggregate(list(Inventory2$colour), length)
print(agg_result)
```

```
##   Group.1 style colour size price discount price_category
## 1   black   44    44   44   44      44              44
## 2   blue   44    44   44   44      44              44
## 3  green   38    38   38   38      38              38
## 4    red   36    36   36   36      36              36
## 5  white   38    38   38   38      38              38
```

```
# Berechnung des durchschnittlichen Preises pro Farbe
avg_price_by_color <- aggregate(price ~ colour, data = Inventory2, FUN = mean)
print(avg_price_by_color)
```

```
##   colour    price
## 1  black  98.18182
## 2   blue  98.18182
## 3  green 108.68421
## 4   red 100.69444
## 5  white 101.71053
```

Mehrere Aggregationen gleichzeitig mit dplyr

```
library(dplyr)
summary_by_color <- Inventory2 |>
  group_by(colour) |>
  summarise(
    count = n(),
    avg_price = mean(price),
    min_price = min(price),
    max_price = max(price),
    total_value = sum(price)
  )
print(summary_by_color)
```

```
## # A tibble: 5 x 6
##   colour count avg_price min_price max_price total_value
##   <chr> <int>   <dbl>     <dbl>    <dbl>      <dbl>
## 1 black     44    98.2        70      135        4320
## 2 blue      44    98.2        70      135        4320
## 3 green     38   109.         75      135        4130
## 4 red       36   101.         70      135        3625
## 5 white     38   102.         70      130        3865
```

Statistische Erklärung: Aggregation ist der Prozess, bei dem Daten gruppiert und zusammengefasst werden, um Muster und Trends auf einer höheren Ebene zu erkennen. Typische Aggregationsfunktionen sind:

- **Zählen:** Anzahl der Beobachtungen in jeder Gruppe
- **Summe:** Gesamtsumme der Werte in jeder Gruppe
- **Mittelwert:** Durchschnittswert in jeder Gruppe
- **Median:** Mittlerer Wert in jeder Gruppe
- **Minimum/Maximum:** Kleinster/grösster Wert in jeder Gruppe
- **Standardabweichung:** Mass für die Streuung in jeder Gruppe

Aggregation ist ein zentraler Bestandteil der deskriptiven Statistik und der explorativen Datenanalyse, da sie komplexe Datensätze auf interpretierbare Kennzahlen reduziert.

R-Erklärung:

- Die Funktion `aggregate()` in Base R gruppiert Daten und wendet eine Funktion auf jede Gruppe an.
- Die Formel-Notation `price ~ colour` bedeutet "aggregiere price nach colour".
- Das Paket `dplyr` bietet mit `group_by()` und `summarise()` eine intuitivere und flexiblere Methode zur Aggregation.
- Die Funktion `n()` in `dplyr` zählt die Anzahl der Beobachtungen in jeder Gruppe.
- Mit `summarise()` können mehrere Aggregationen gleichzeitig durchgeführt werden.

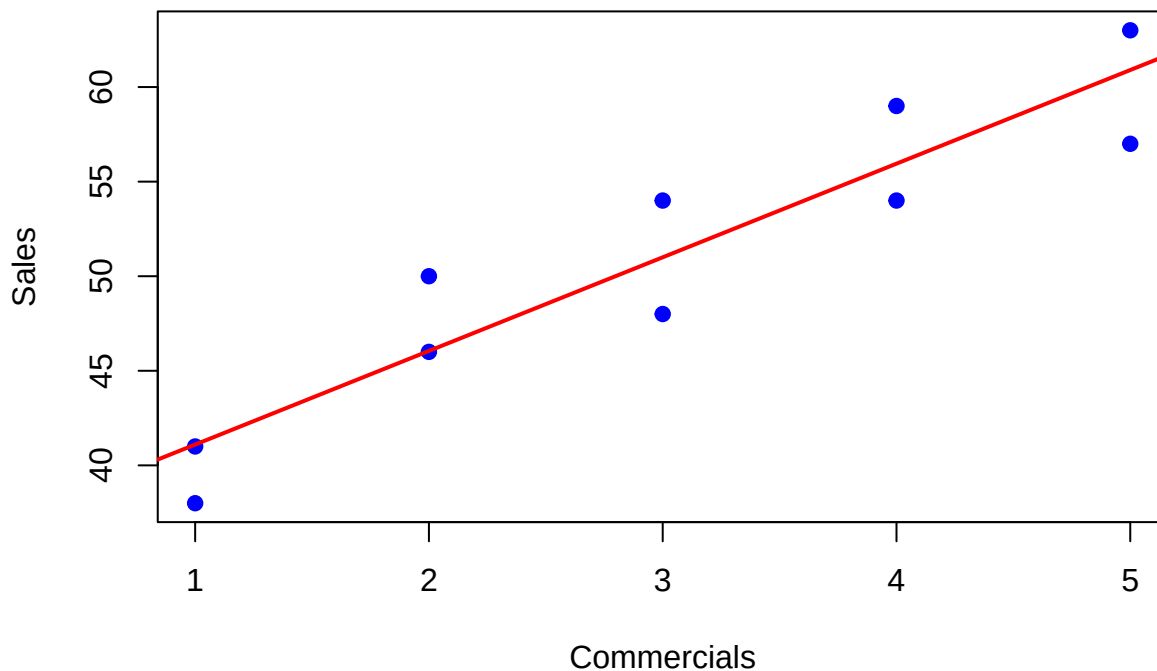
14 Streudiagramme

Streudiagramme visualisieren die Beziehung zwischen zwei quantitativen Variablen.

```
# Erstellen eines Streudiagramms
plot(Sales ~ Commercials,
     data = Stereo,
     main = "Streudiagramm: Sales vs Commercials",
     xlab = "Commercials",
     ylab = "Sales",
     pch = 19,                      # Punktsymbol (gefüllter Kreis)
     col = "blue")

# Hinzufügen einer Trendlinie mittels linearer Regression
fit <- lm(Sales ~ Commercials, data = Stereo) # Lineares Regressionsmodell
abline(fit, col = "red", lwd = 2)           # Regressionslinie hinzufügen
```

Streudiagramm: Sales vs Commercials



```
# Anzeigen der Modellzusammenfassung
summary(fit)

##
## Call:
## lm(formula = Sales ~ Commercials, data = Stereo)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.900 -2.737 -0.075  2.775  3.950
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   36.150     2.285   15.820 2.55e-07 ***
## Commercials    4.950     0.689    7.185 9.39e-05 ***
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.081 on 8 degrees of freedom
## Multiple R-squared:  0.8658, Adjusted R-squared:  0.849
## F-statistic: 51.62 on 1 and 8 DF,  p-value: 9.386e-05
```

```
# Erweitertes Streudiagramm mit ggplot2
```

```
library(ggplot2)
```

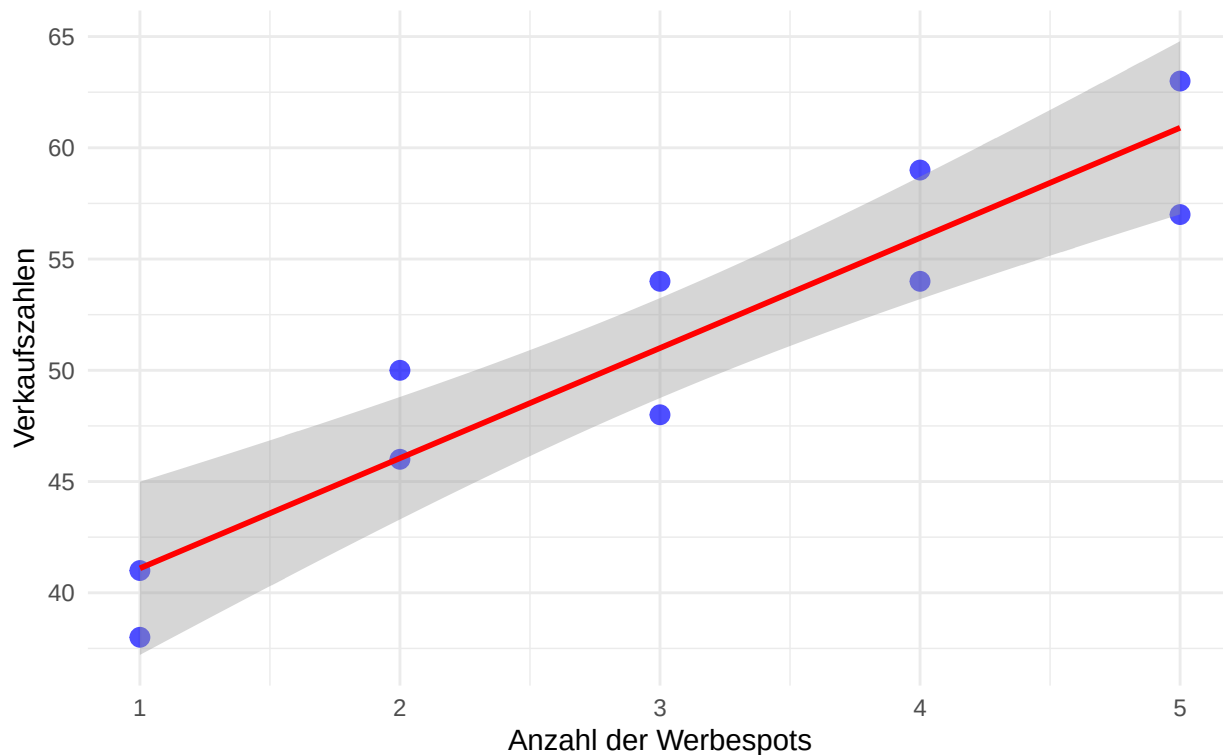
```
##
## Attaching package: 'ggplot2'
## The following object is masked from 'package:element':
##
##     element
```

```
ggplot(Stereo, aes(x = Commercials, y = Sales)) +
  geom_point(color = "blue", size = 3, alpha = 0.7) +
  geom_smooth(method = "lm", color = "red") +
  labs(title = "Verkäufe vs. Werbung",
       subtitle = "Mit Konfidenzintervall für die Regressionslinie",
       x = "Anzahl der Werbespots",
       y = "Verkaufszahlen") +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

Verkäufe vs. Werbung

Mit Konfidenzintervall für die Regressionslinie



Statistische Erklärung: Streudiagramme (Scatterplots) sind eine grundlegende Methode zur Visualisierung der Beziehung zwischen zwei kontinuierlichen Variablen. Jeder Punkt im Diagramm repräsentiert eine Beobachtung, wobei die Position des Punktes durch die Werte der beiden Variablen bestimmt wird.

Streudiagramme helfen dabei:

- Die Art der Beziehung zu erkennen (linear, nicht-linear)
- Die Stärke der Beziehung einzuschätzen
- Muster, Cluster und Ausreisser zu identifizieren

Die lineare Regression ist eine statistische Methode, um den linearen Zusammenhang zwischen zwei Variablen zu modellieren. Die Regressionslinie minimiert die Summe der quadrierten vertikalen Abstände zwischen den Datenpunkten und der Linie.

R-Erklärung:

- Die Funktion `plot()` mit der Formel-Notation `y ~ x` erstellt ein Streudiagramm.
- Parameter für `plot()`:
 - `pch`: Plot character (Symbol für die Punkte)
 - `col`: Farbe der Punkte
 - `main`, `xlab`, `ylab`: Titel und Achsenbeschriftungen
- Die Funktion `lm()` (linear model) berechnet eine lineare Regression.
- Die Funktion `abline()` fügt eine Linie zu einem bestehenden Plot hinzu.
- Die Funktion `summary()` gibt eine detaillierte Zusammenfassung des Regressionsmodells aus.
- Das Paket `ggplot2` bietet eine alternative, leistungsstarke Methode zur Erstellung von Grafiken.

Hinweis zur Interpretation: Die Steigung der Trendlinie gibt Aufschluss über die Art des Zusammenhangs:

- Positive Steigung: positive Beziehung (wenn x steigt, steigt auch y)
- Keine Steigung: kein Zusammenhang

- Negative Steigung: negative Beziehung (wenn x steigt, sinkt y)

Der R^2 -Wert (im `summary(fit)` als “Multiple R-squared” angegeben) gibt an, welcher Anteil der Varianz in der abhängigen Variable durch das Modell erklärt wird. Ein höherer Wert deutet auf einen stärkeren linearen Zusammenhang hin.